

CS 3300: Intro to software Engineering

Final Project Presentation
Software Development Report Briefing

Project Title: Vibe Vision
Date: 4/23/26

Department of Computer Science
University of Colorado at Colorado Springs



TEAM

Team Members and Roles

- Team A
 - Joshua – Github Maintainer + Kotlin dev
 - Lydia – Documenter + Tester
- Team B
 - Syndeed – ML dev + Documenter
 - Jared – Interfacing developer + Testing



PROBLEM STATEMENT

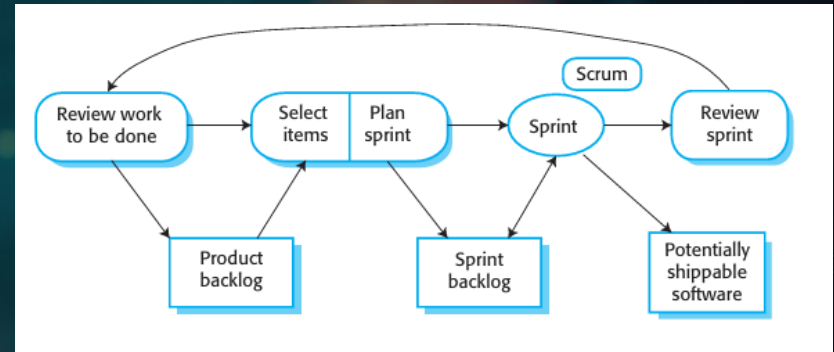
Some software applications rely on understanding user context to provide meaningful, adaptive experiences. There is a need for an accessible, on-device method to infer basic emotions – such as happiness, sadness, anger, or neutrality – directly from camera input so that applications can modify their behavior without relying on external services that could compromising user privacy

PROJECT INTRODUCTION

- Project Title: **Vibe Vision**
- Project Description:
 - We developed an Android application that was built upon a pre-existing machine learning model to determine whether the user appears happy, sad, angry, or neutral based on an image captured through the device's camera. The app processes facial expressions in real time, classifies emotional states using a trained image-based ML classifier, and provides feedback within the user interface.
 - Python ML model with Kotlin Android App UI with Tensorflow Lite interface

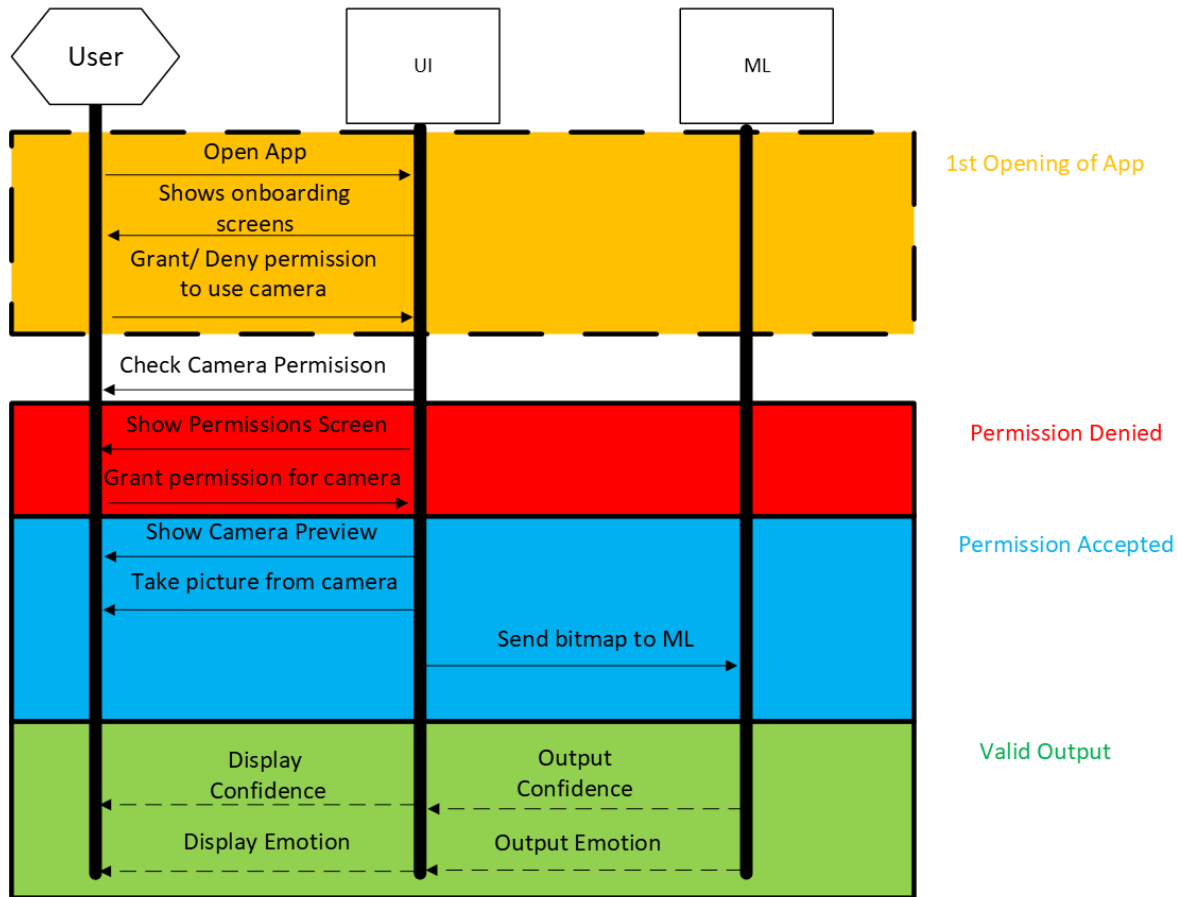
SOFTWARE ENGINEERING PRINCIPLES

- **Agile** because requirements were flexible and we wanted continuous feedback and reevaluation.
- **Why Agile?**
 - Requirements changed
 - Fast development needed
 - Team feedback is important
- **Why Scrum?**
 - Clear structure
 - Defined roles
 - Short development cycles (Sprints)
 - Easy progress tracking

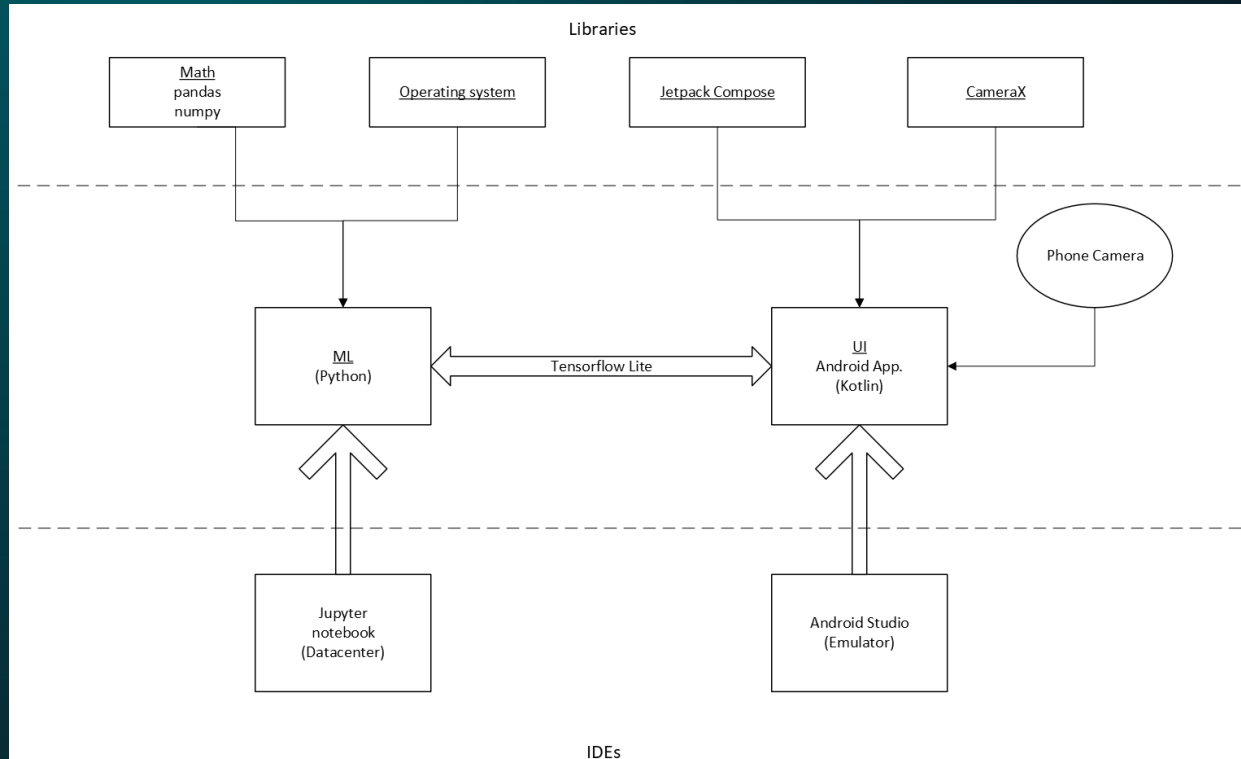


Scrum Diagram from Software Engineering by Ian Sommerville

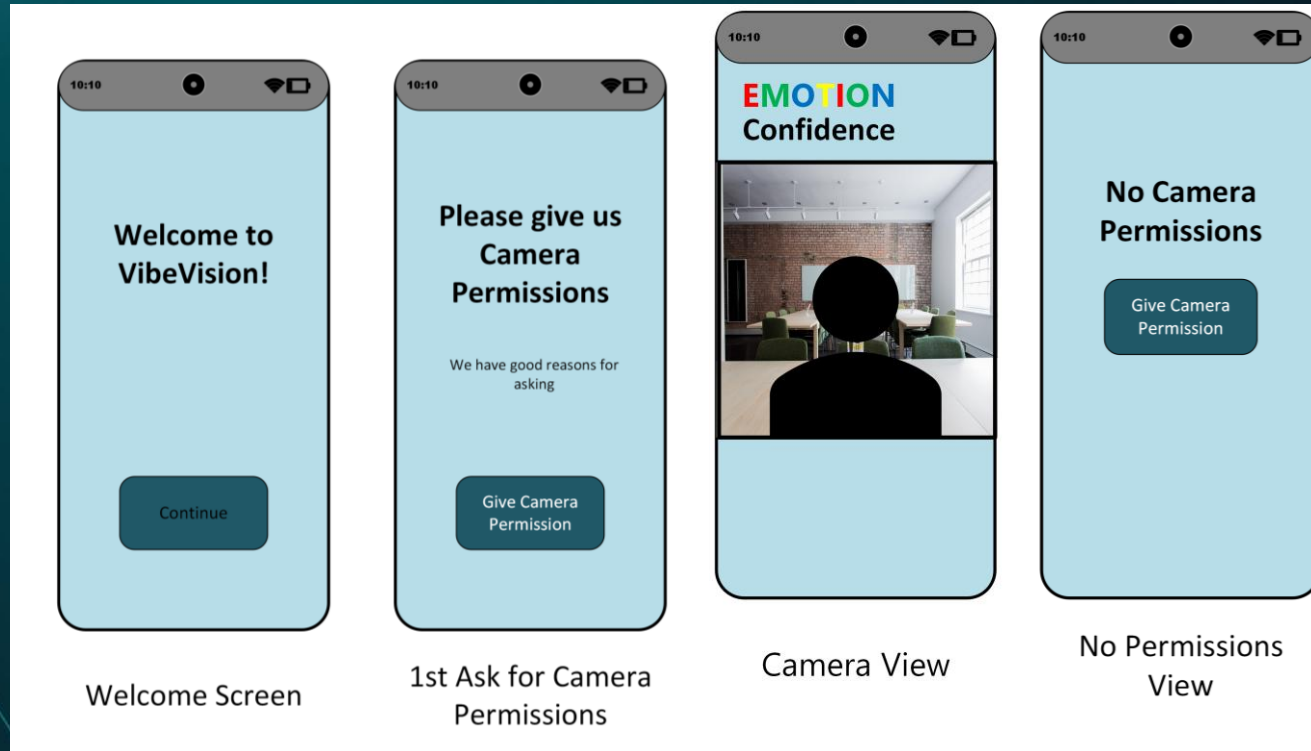
PROJECT ARCHITEC- TURE DIAGRAM



PROJECT SOFTWARE DESIGN DIAGRAM



PROJECT PERSPECTIVE MODEL DIAGRAM



Schedule

Planned

Sprint 1

Iteration 0 – Establish ML & App Workspace

Iteration 1 – Establishing Interfaces

Iteration 2 – Adding Complexity to
Interfacing

Iteration 3 – Establish MVP

Integration testing

Sprint 2 and 3

Yeah, right

Artifact/ Documentation

Test report, users manual, etc.

Code

Source Code, License, UCCS TA software
read-me file

Actual

Sprint 1

Iteration 1 – Data prep & organization –
basic camera access

Iteration 2 – basic ML + camera access

Iteration 3 – Model Training,
Experimentation & Integration

Iteration 4 – ML finalization – image
processing in app

Iteration 5 – Integration & cosmetics

Artifact/ Documentation

Test report, users manual, etc.

Code

Source Code, License, UCCS TA software
read-me file

PROJECT FUNCTIONAL REQUIREMENTS

Planned	Actual
REQ-01: App opens and requests webcam permission	REQ-01: App opens and requests webcam permission
REQ-02: Webcam captures 720p images	REQ-02: Webcam captures 128*128 images
REQ-03: Face detection works (90% dataset / 70% app)	REQ-03: Face detection works (90% dataset / 70% app)
REQ-04: Returns “no face detected”	REQ-04: Returns “no face detected”
REQ-05: Expression classification works (60% dataset / 40% app)	REQ-05: Expression classification works (60% dataset / 40% app)
REQ-06: Returns confidence score	REQ-06: Returns confidence score
REQ-07: Invalid emotion throws error	REQ-07: Invalid emotion throws error
REQ-08: App sends image to ML and receives result	REQ-08: App sends image to ML and receives result
REQ-09: UI displays emotion, face detection, confidence	REQ-09: UI displays emotion, face detection, confidence
REQ-10: App closes and turns off webcam	REQ-10: App closes and turns off webcam
REQ-11: Runs on desktop Kotlin JVM	REQ-11: Runs on desktop Kotlin JVM
REQ-12: Runs offline (no network)	REQ-12: Runs offline (no network)
REQ-13: Does not store user images	REQ-13: Does not store user images
REQ-14: App opens in < 30 seconds	REQ-14: App opens in < 30 seconds
REQ-15: Emotion computed in < 1 minute	REQ-15: Emotion computed in < 1 minute

PROJECT VERIFICATION

- How many tests did you perform?
 - 24 final tests
 - We tested the app continually while developing it.
 - Integration tests were mostly conducted near the end of the sprint.
- How many issues/bugs/flags did you average per test?
 - 8.3% tests failed
- How many issues/bugs/flags/faults remain open?
 - 2 issues remain open
- Accuracy of emotions is low, below what was required. We ran the ML with the test data and got an accuracy of: 53%

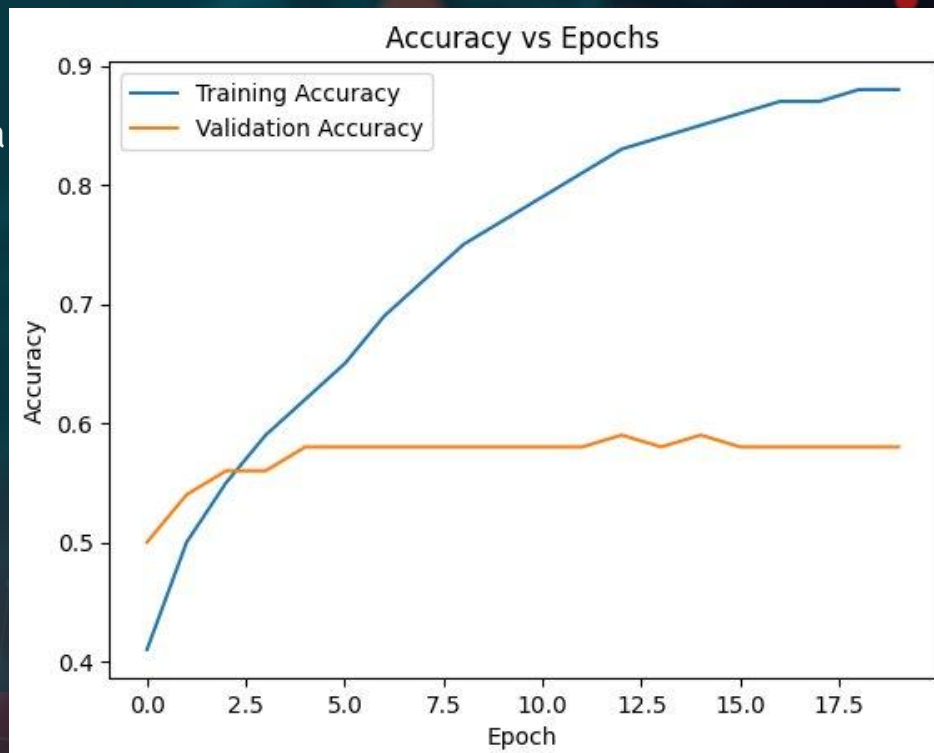
PROJECT CONFIGURATION MANAGEMENT

- Hardware Configuration:
 - GPU: A100 GPU (Google Collab)
 - GPU RAM: 80GB
 - System RAM: 168GB
- Model configuration:
 - Model architecture: Sequential Convolutional Neural Network
 - Input format: 128*128*1 (greyscale)
 - Loss function: sparse_categorical_crossentropy
 - Optimizer: Adam
 - Batch Size: 32 of 128x128 images

Training and Validation Performance

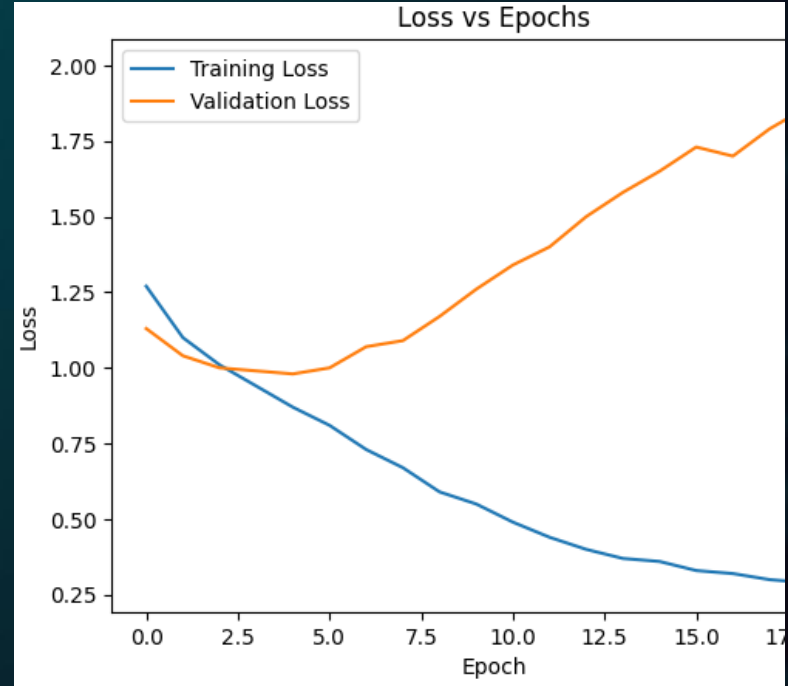
Training:

- Model trained for **100 epochs**
- Training accuracy increased from **~41% → ~95.7%**
- Model successfully learned patterns from training data
- Continuous improvement across epochs with no early stopping
- Validation:
 - Validation accuracy improved early, then plateaued:
 - Peak around **~59–60%**
 - Final accuracy: **~58%**
- Declining performance on unseen data



Loss over the epochs

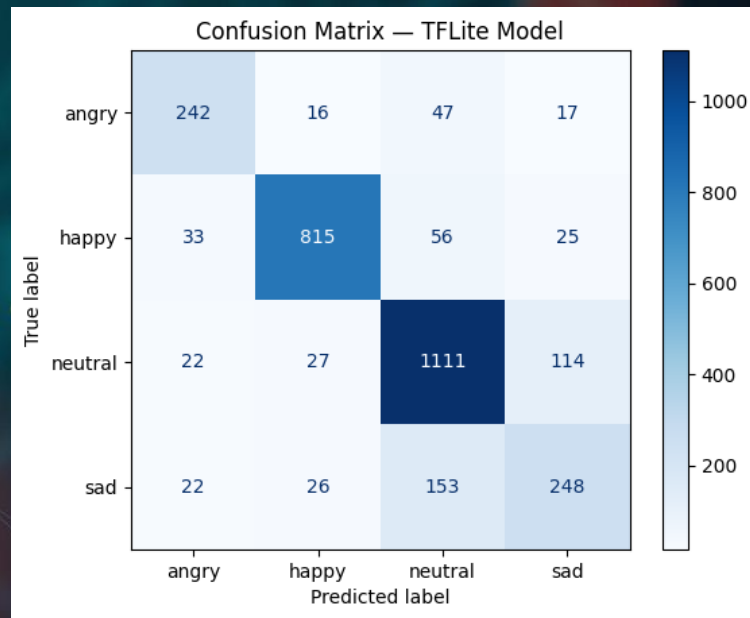
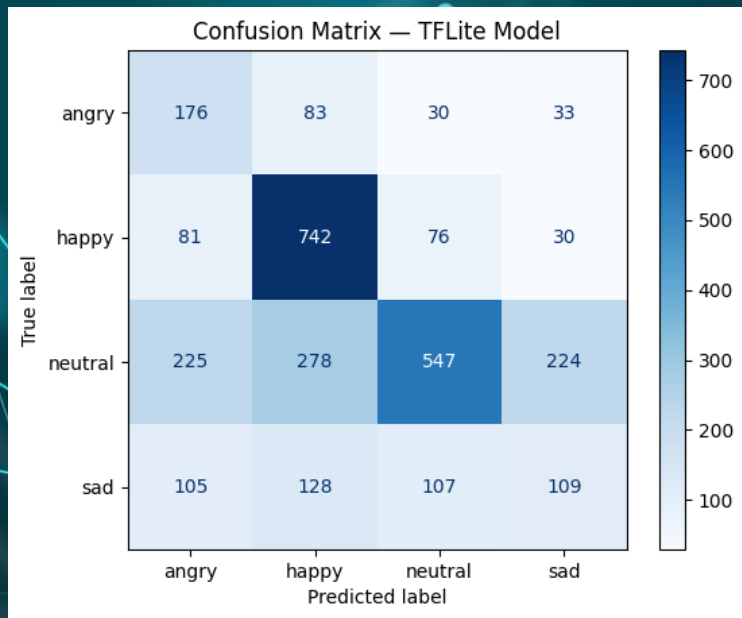
- Training loss steadily decreases from $\sim 1.27 \rightarrow \sim 0.11$
- Model is improving fit on training data
- Validation loss initially decreases ($\sim 1.13 \rightarrow \sim 0.98$)
- After early epochs, validation loss starts increasing continuously
- Final validation loss reaches ~ 3.7
- Clear divergence between training and validation loss curves



Key Observations

- Overfitting:
- Large gap between training and validation accuracy:
 - Training: ~**95%**
 - Validation: ~**58%**
- Training loss decreases while validation loss increases
- Model memorizes training data instead of generalizing
- Model has **high variance but isn't biased**
- Learns noise and specific patterns instead of general features
- Validation data contains unseen variations and performance drops
- Overfitting starts around **Epoch 6–10**
- Training loss goes down but validation loss increases

ML Test Graph



PROJECT SOFTWARE DEVELOPMENT

- Open-source utilities: libraries
 - Python: TensorFlow, Keras, NumPy,
 - Kotlin: Jetpack Compose, CameraX
- Java Development Kit (JDK) version: 17
- Software Development Kit (SDK) version: android 36.1
- Python 3.12.7
- Gradle 9.2.1
- Java 11
- Android Studio
- Jupyter Notebook

ETHICS

- Avoiding Ethical Pitfalls
 - We ensured no misuse of data, maintaining user privacy, and avoiding biased or misleading model behavior.
- Modifications to Requirements
 - Some requirements were refined and clarified during development to better align with technical constraints and project scope. Old requirements were kept, and the reasons for dropping them were documented.
- Removal of Requirements
 - A small number of lower-priority requirements were removed or deferred to reduce unresolved bugs and ensure a stable, functional final product.

RISK MANAGEMENT

- Environment stability: manually setup procedure for each member of the team. Works for small teams
 - Learned not to update dependencies during development unless necessary
- Risk tracking and management
 - Integration of UI and ML tracked through development
 - Discord to coordinate “work from home” days
- How was your time management?
 - Time Management was good before spring break but poor exiting the break

PROJECT CONFIGURATION MANAGEMENT

- Configuration Management Activities
 - Version control maintained using Git (branching, commits, and merges)
 - Backup and version history maintained for all major artifacts
 - Test results and documentation updated alongside code changes
- Final Configuration Metrics
 - Lines of Code: ~1500 lines across ML and application layers
 - Code Size: ~37 MB on device including model files and dependencies
 - Run Time:
 - ML inference: ~0.5 per image
 - App response time: 3 seconds to open
- Number of modules: 3: ML, interface, UI
- Test coverage: Unit and integration testing across all major components
- Build success rate: High with minor integration issues resolved

PROJECT COMPLETION

- What is the final output of your project?
 - An android app that detects emotions based on real time camera input. A minimal viable product to visualize the concept
- Identify finished state
 - Open-source, MIT licensed

Future development

- Set up app for Google Play Store and test on other Android devices
- Keep training ML on larger dataset
- Implement new features
 - "Journal" of emotions of day
 - flip to front camera
 - Graphic scale for confidence level
 - Add more emotion labels: Disgust, fear, etc.
- Support for multiple faces and facial detection
 - No face detected state
- Standardize App Permission screens

Project Demonstration

- Example on Emulator
- Question?